07 — Native Binary Pipeline: gcc, Linking, and the Entry Shim

Overview

The native binary pipeline converts a JOCKY source file into a standalone Windows `.exe` that runs without any Python or LLVM runtime. The pipeline involves four steps:

```
hello.jk
    │
    ▼ (JOCKY compiler — Stages 1–5)
    │
    ▼
hello.o                    ← JOCKY's compiled code (no entry point, unresolved
externals)
    │
    ├── forensics.o         ← stdlib: report(), procs_list(), etc.
    │   (pre-built by build_stdlib.py)
    │
    ├── _jocky_entry.o      ← tiny shim: int main() { start(); return 0; }
    │   (auto-generated per compile)
    │
    ▼  gcc links all three
    │
    ▼
hello.exe                  ← standalone native binary
```

What Is an Object File?

An object file (`.o` on Linux/MinGW, `.obj` on MSVC) is the output of a compiler before linking. It contains:

Code section (`.text`)

The compiled machine code for every function defined in the source. Instructions are fully compiled for the target CPU (x86-64) but function calls to external symbols use placeholder values.

For `hello.o`:

```
Function: start()
  0000:  push rbp
  0001:  mov rbp, rsp
  0004:  lea rdi, [rip + ???]    ← "???" = relocation for .jk_str.0
  000b:  call ???                 ← relocation for report()
  0010:  pop rbp
  0011:  ret
```

The `???` placeholders are filled in by the linker.

Data sections (`.data`, `.rodata`, `.bss`)

Global variables and constants. String literals go into `.rodata` (read-only data). The `_jocky_build_id`, `_jocky_entropy`, and `.jk_str.*` globals from the IR all land here.

Symbol table

A list of names defined or referenced:

```
DEFINED (exported):  start
UNDEFINED (external): report, procs_list, proc_count, proc_name, proc_kill,
                      strcmp, _jocky_build_id, _jocky_entropy, .jk_str.0
```

`start` is defined — the linker can locate its code. `report` is undefined — the linker must find its definition in another object file (in this case, `forensics.o`).

Relocation table

A list of every location in the code or data section that needs to be patched with a resolved address:

```
Offset 0x0004 in .text: replace ??? with address of .jk_str.0 in .rodata
Offset 0x000b in .text: replace ??? with address of report() in forensics.o
```

What `build_stdlib.py` Does

```
cmd = [GCC, '-c', 'stdlib/forensics.c', '-o', 'stdlib/forensics.o',
       '-O2', '-std=c11', '-Wall']
```

- `-c` : compile only, do not link. Produces a `.o` file.
- `-O2` : optimise (inline small functions, eliminate dead code).
- `-std=c11` : use C11 standard.

`forensics.o` has:

- `report()`, `procs_list()`, `proc_count()`, etc. as **defined** symbols
- `printf`, `malloc`, `fflush`, etc. as **undefined** (resolved against the C runtime library at final link time)

You only need to run `build_stdlib.py` once, or whenever you modify `forensics.c`.

The Entry Shim Problem

JOCKY's entry function is named `start`. A standard Windows executable expects `main` (for console programs) or `WinMain` (for GUI programs) as the entry point. The C runtime (`crt0.o`, included by gcc automatically) calls `main()` after initialising the runtime.

If we link `hello.o` directly, gcc looks for `main()`, finds none, and reports:

```
undefined reference to `WinMain'
```

(or `main`, depending on the gcc version and settings).

Solution: auto-generate a tiny C file every compilation:

```
entry_c = os.path.join(output_dir, '_jocky_entry.c')
with open(entry_c, 'w') as f:
    f.write('extern void start(void);\nint main(void){start();return 0;}\n')
```

Content of `_jocky_entry.c`:

```
extern void start(void);
int main(void) { start(); return 0; }
```

- `extern void start(void)` — declares that `start()` exists somewhere (it's in `hello.o`)
- `int main(void)` — the entry point gcc and the C runtime expect
- `{ start(); return 0; }` — call the JOCKY program, then exit cleanly

This file is compiled with:

```
subprocess.run([GCC, '-c', entry_c, '-o', entry_o, '-O2'], ...)
```

The resulting `_jocky_entry.o` has `main` as a defined symbol and `start` as an undefined symbol.

The Link Step

```
link_cmd = [
    GCC,
    obj_path,       # hello.o       — JOCKY compiled code (has 'start')
    forensics_o,    # forensics.o   — stdlib (has 'report', 'procs_list', etc.)
    entry_o,        # _jocky_entry.o — entry shim (has 'main', needs 'start')
    '-o', exe_path, # hello.exe     — output
    '-O2',          # optimise
    '-mconsole',    # console subsystem (entry via 'main', not 'WinMain')
]
```

gcc invokes the linker (`ld`) with all three object files plus the MinGW runtime libraries (implicitly added). The linker:

1. Reads all symbol tables: knows what each `.o` defines and needs
2. Resolves all undefined symbols:
   - `main` → defined in `_jocky_entry.o`
   - `start` → defined in `hello.o`
   - `report` → defined in `forensics.o`
   - `procs_list` → defined in `forensics.o`
   - `strcmp` → defined in MinGW's `msvcrt.dll` import library
   - `printf`, `malloc` → same
3. Applies all relocations: patches the `???` placeholders with real addresses
4. Writes the PE (Portable Executable) `.exe` file

`-mconsole` tells the linker to set the PE subsystem flag to `IMAGE_SUBSYSTEM_WINDOWS_CUI` (3 = console). Without this, MinGW defaults to `IMAGE_SUBSYSTEM_WINDOWS_GUI` (2), which expects `WinMain` instead of `main`.

The PIC / Static Relocation Issue

What is PIC?

Position-Independent Code (PIC) is code that works correctly regardless of where it is loaded in memory. Shared libraries (`.dll`) must use PIC because multiple programs load the same DLL at potentially different base addresses.

PIC accesses global data through the GOT (Global Offset Table) — a table of pointers that is filled in by the dynamic linker when the DLL is loaded.

Why PIC causes problems for exe linking

Windows executables are position-dependent by default — they have a preferred load address and the loader relocates them if necessary, but they don't use a GOT.

LLVM's default relocation model for x86-64 on Windows produces PIC references. When the linker tries to build an executable using PIC object files, it encounters GOT references it cannot resolve:

```
undefined reference to `_GLOBAL_OFFSET_TABLE_'
```

The fix

```
tm = target.create_target_machine(opt=2, reloc='static')
```

`reloc='static'` tells LLVM to emit PDC (Position-Dependent Code) — code that uses direct absolute or PC-relative addressing for globals, with no GOT indirection. This is what the MinGW executable linker expects.

What the Final `.exe` Contains

After linking, `hello.exe` is a Windows PE (Portable Executable) file with:

PE Header: Describes the file type (executable), target machine (x86-64), subsystem (console), entry point address, and section layout.

`.text` **section:** All function code from all three `.o` files:

- `start()` from `hello.o`
- `report()`, `procs_list()`, etc. from `forensics.o`
- `main()` from `_jocky_entry.o`

`.rdata` **section:** Read-only data:

- String literals (encrypted, if obfuscation was applied)
- `_jocky_build_id` and `_jocky_entropy` globals
- Import address table (pointers to DLL functions)

`.data` **section:** Writable data (minimal — JOCKY doesn't use writable globals)

Import Directory: Lists DLL dependencies:

- `msvcrt.dll` — provides `printf`, `malloc`, `strcmp`, `fflush`, etc.

No dependency on Python. No dependency on LLVM. The executable is standalone.

Running the Final Binary

```
.\output\hello.exe
[JOCKY] Hello from JOCKY!
[JOCKY] JOCKY compiler is working.
```

When Windows loads `hello.exe`:

1. Maps the PE sections into memory
2. Resolves imports (finds `msvcrt.dll`, patches import address table)
3. Jumps to the entry point (which calls `main()`)
4. `main()` calls `start()`
5. `start()` calls `report()` which calls `printf()`
6. Output appears on screen
7. `main()` returns 0 → clean exit

Size and Performance

A typical JOCKY hello-world binary is approximately 83 KB. This includes:

- The compiled code (< 1 KB)
- The forensics.o code (~20 KB)
- The MinGW CRT startup code (~60 KB)
- PE headers and padding

Execution time: near-instantaneous. The compiled code runs at full native CPU speed — there is no interpreter, no GIL, no runtime overhead.

Compilation Mode Comparison

Aspect	**JIT mode (`--run`)**	**Native binary (default)**
Requires Python	Yes	No (after compiling)
Requires gcc	No	Yes (for linking)
Time to first output	~1 second (JIT compile)	~2 seconds (compile + link)
Output file	None	`.exe` (runs standalone)
Obfuscation applied	No	Yes
Strings readable	Yes	No (encrypted)
SHA-256 changes each build	N/A	Yes
Suitable for	Development, testing	Deployment, demo